МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии

и информационных технологий

КУРСОВОЙ ПРОЕКТ

по дисциплине

«Современные операционные системы и базы данных»

на тему «Разработка сайта-визитки

фуллстек (фронтенд-фуллстек) разработчика»

Выполнил: обучающийся 2 курса

группы 12-25РПм, очной формы обучения

Трифонов Дмитрий Сергеевич

Направление подготовки 09.04.04 Программная инженерия

Профиль «Разработка программно-информационных систем»

Руководитель: Галлини Надежда Игоревна

Оценка _____ Дата защиты _____

Подпись руководителя _____

Ялта, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ САЙТА-ВИЗИТКИ 5
1.1 Сайт-визитка как форма цифрового представления специалиста 5
1.2 Клиент-серверная модель и роль операционной системы 5
1.3 Хранение данных в веб-приложениях 6
1.4 Варианты расширения: файл SQLite или JSON для каталога проектов и услуг 7
2 ПРОЕКТИРОВАНИЕ САЙТА-ВИЗИТКИ 10
2.1 Требования к функциональности и качеству 10
2.2 Выбор технологического стека 10
2.3 Информационная архитектура и интерфейс 11
3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РАЗВЁРТЫВАНИЕ 14
3.1 Структура проекта и основные компоненты 14
3.2 Взаимодействие с операционной средой хостинга 15
3.3 Служебные файлы robots.txt и sitemap.xml на боевом сайте 16
3.4 Клиентское хранение настроек и вычисляемые данные 17
3.5 Тестирование и результаты 20
ЗАКЛЮЧЕНИЕ 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 22

ВВЕДЕНИЕ

В профессиональной среде первого впечатления часто достаточно нескольких секунд просмотра персональной страницы. Прежде чем состоится разговор или переписка, заинтересованная сторона уже успевает оценить, как специалист оформляет своё присутствие в сети, насколько ясно изложены сильные стороны и насколько удобно начать контакт. Компактная персональная страница, которую принято называть визиткой, как раз и решает эту задачу: она собирает ключевые сведения в одном экране и задаёт визуальный тон.

Если речь идёт о фуллстек (фронтенд-фуллстек) разработчике, ценность такого ресурса возрастает. Страница одновременно показывает практический навык вёрстки и клиентской логики, аккуратность в подготовке к выкладке и умение держать интерфейс понятным на разных устройствах. Даже когда объём контента невелик, приходится продумать иерархию блоков, скорость открытия, устойчивость вёрстки и порядок файлов на сервере.

При этом в рабочей визитке акцент сделан на роли фронтенд-специалиста: автор позиционирует себя как фуллстек (фронтенд-фуллстек) разработчик — может выполнять и серверную часть, но специализируется на клиенте. Поэтому интерфейс и клиентская логика вынесены на первый план, а серверная динамика и СУБД для данной одностраничной визитки не требуются.

Тема курсовой работы актуальна потому, что веб-страница существует не «сама по себе», а внутри связки «браузер - сеть - операционная среда хостинга». Для дисциплины «Современные операционные системы и базы данных» здесь важны как минимум три аспекта: как ОС обслуживает выдачу файлов, где физически лежат данные и чем клиентское хранилище отличается от серверной СУБД. Отдельно учитывается подготовка ресурса к индексации.

Цель работы - создать и подробно описать персональную визитку фуллстек (фронтенд-фуллстек) разработчика, пояснив выбранный стек и схему размещения в сети.

Чтобы достичь цели, последовательно решаются следующие задачи:

1. выяснить, какие ожидания предъявляются к персональной веб-визитке специалиста;

2. разобрать, как браузер и сервер взаимодействуют при открытии страницы и какую роль здесь играет ОС;

3. сопоставить варианты хранения сведений: файлы на диске, браузерное хранилище и классическая СУБД;

4. спроектировать и собрать одностраничный интерфейс с адаптацией и умеренной интерактивностью;

5. выложить результат на хостинг и оформить в разметке минимальный набор средств поисковой подготовки (title, description, canonical, Open Graph, Twitter Card, JSON-LD).

Объект исследования - создание небольшого персонального веб-ресурса специалиста.

Предмет исследования - приёмы построения сайта-визитки фуллстек (фронтенд-фуллстек) разработчика с опорой на свойства операционных систем и способы хранения данных.

В работе применяются изучение источников и документации, сопоставление вариантов архитектуры, макетирование интерфейса, практическая реализация, проверка в браузерах и публикация на хостинге.

Практический итог - действующая визитка по адресу https://trifonix.github.io/trifonix-personal-website-prototype/. Полученные решения можно переносить на другие лёгкие персональные страницы.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ САЙТА-ВИЗИТКИ

1.1 Сайт-визитка как форма цифрового представления специалиста

Под сайтом-визиткой понимают небольшой публичный ресурс, где человек или организация кратко представляются аудитории [1]. У него нет задач полноценного портала или торговой площадки: не нужны корзина, личный кабинет и сложные сценарии. На первый план выходят ясная подача, быстрый отклик страницы и понятный путь к контакту.

В персональной визитке разработчика обычно встречаются:

– блок идентичности: ФИО, снимок, должность или роль;

– короткое резюме умений;

– сведения о местах работы и учёбе;

– список технологий;

– выходы на проекты и мессенджеры.

С инженерной позиции такой ресурс ближе к информационной витрине, чем к транзакционной системе. Контент меняется редко, поэтому его удобно держать в разметке. Вместе с тем на клиенте допустимы небольшие сценарии - смена оформления, подсчёт стажа, анимация акцентов. Главное остаётся прежним: показать сведения, а не обслуживать поток операций.

Для фуллстек (фронтенд-фуллстек) разработчика визитка ещё и «пробный стенд» полного пути поставки: сверстать, оформить стилями, добавить логику, положить файлы на сервер, проверить служебные robots.txt и sitemap.xml, убедиться в корректном виде на телефоне и мониторе. В итоге страница работает и как продукт, и как фрагмент портфолио.

1.2 Клиент-серверная модель и роль операционной системы

Сеть веб-ресурсов опирается на разделение ролей клиента и сервера [2]. Клиент - это браузер: он отправляет запрос по HTTP или HTTPS, получает ответ, разбирает разметку, применяет таблицы стилей, запускает сценарии и рисует интерфейс. Сервер принимает обращение, находит нужный файл либо формирует ответ программно и возвращает его обратно.

ОС на стороне сервера создаёт условия для работы веб-сервиса (часто Apache либо Nginx): поднимает процессы, контролирует права на каталоги, открывает сетевые порты, ведёт журналы, обслуживает сертификаты [3]. Даже если страница полностью статическая, без корректной настройки ОС нельзя гарантировать безопасный доступ к публичным файлам и стабильную выдачу.

На устройстве пользователя своя ОС (Windows, macOS, Android, iOS и другие) запускает браузер как приложение, выделяет ему память и сеть, ограничивает доступ к системе. Внутри браузера действуют собственные правила: исполнение JavaScript преимущественно в одном потоке интерфейса, политика same-origin, лимиты на объём localStorage. Эти ограничения нужно учитывать уже на этапе проектирования.

Открытие визитки обычно проходит так:

1. человек набирает адрес или переходит по ссылке;
2. служба имён преобразует домен в IP;
3. устанавливается защищённое соединение;
4. сервер под управлением ОС отдаёт index.html и связанные файлы;
5. браузер строит дерево документа, накладывает стили и выполняет скрипты.

Зная эту цепочку, проще выбрать площадку размещения: shared-хостинг, VPS или чисто статический хост. В курсовом проекте использован статический хостинг GitHub Pages: ОС и веб-сервер предоставляет площадка, а автор отвечает за наполнение открытого каталога репозитория.

1.3 Хранение данных в веб-приложениях

СУБД нужны там, где сведения нужно систематически сохранять, искать и изменять [4]. В типичном динамическом сайте серверная база (PostgreSQL, MySQL и аналоги) держит пользователей, материалы и события; приложение читает и пишет её через программный слой, а ОС обеспечивает процессы СУБД и доступ к дискам.

Для визитки отдельный серверный склад часто избыточен: тексты и изображения зафиксированы и могут жить прямо в файлах проекта. При этом хранение всё равно проявляется в двух плоскостях.

На сервере файловая система ОС хранит страницу, иллюстрации, иконки, манифест и SEO-служебники. Здесь важны порядок каталогов, типы содержимого и правила кэша.

В браузере доступен Web Storage. Через localStorage можно оставлять строковые пары ключ-значение между визитами [5]. В отличие от cookie, эти записи не уезжают на сервер с каждым запросом. Реляционных выборок, транзакций и совместного доступа нескольких пользователей там нет, зато удобно помнить личные настройки оформления.

В выполненном проекте в localStorage фиксируется выбранная палитра (светлая либо тёмная). Так разделяются:

– содержимое самой визитки, которое сопровождает исходный код;

– предпочтения посетителя, привязанные к его браузеру.

Отдельно стоит отметить машиночитаемые описания Open Graph и JSON-LD. Это не замена СУБД, но способ сообщить поисковикам и мессенджерам, о ком страница и какое изображение к ней относится.

1.4 Варианты расширения: файл SQLite или JSON для каталога проектов и услуг

Текущая визитка намеренно остаётся одностраничным статическим лендингом на высоту окна (около 100vh) в виде «экрана» веб-приложения: объём сведений невелик, контент меняется редко, отдельный сервер и классическая СУБД здесь избыточны. Вместе с тем полезно рассмотреть, как ту же концепцию можно развить, если появятся несколько страниц и потребуется централизованно хранить перечень оказанных услуг или проектов.

Один из лёгких вариантов - один файл базы SQLite рядом с сайтом (например, data/portfolio.sqlite). В нём достаточно двух-трёх таблиц: projects (или services) с полями id, title, description, url, published_at; technologies (name);

связующая таблица project_technologies. SQLite не требует отдельного сервера СУБД: это один файл на диске, который удобно копировать вместе с проектом и отдавать через простой серверный сценарий (Node.js, Python, PHP) либо через serverless-функцию на хостинге.

Если сайт вырастет до нескольких страниц (главная, список проектов, карточка проекта, услуги), подтягивание данных выглядит так. Браузер запрашивает HTML-страницу или API-маршрут вида /api/projects и /api/projects/:id. Серверный слой открывает файл SQLite, выполняет параметризованный SELECT (с JOIN по технологиям при необходимости) и возвращает HTML-фрагмент либо JSON. Карточка проекта на отдельной странице получает идентификатор из адреса и читает одну запись; список на каталоге - набор опубликованных записей с краткими полями. Так сохраняется идея «витрины», но содержимое каталога перестаёт быть «зашитым» в разметку каждой страницы.

Ещё более простой путь без SQL - файл данных JSON (например, data/projects.json) с массивом объектов { "id", "title", "description", "url", "technologies": [...] }. При полностью статической публикации (как сейчас на GitHub Pages) такой файл можно загружать на клиенте через fetch и отрисовывать список скриптом: отдельный сервер не нужен, достаточно положить JSON в репозиторий. Минусы очевидны: нет удобных запросов и ограничений целостности как в SQLite, сложнее атомарно обновлять связанные поля, а правка файла по-прежнему ручная либо через отдельную админ-утилиту. Плюсы - прозрачность, лёгкий дифф в git и полное соответствие текущей статической модели развёртывания.

Сопоставление для данной визитки. JSON уместен как первый шаг к многостраничности без смены хостинга: главная остаётся 100vh-витриной, а /projects.html читает общий файл данных. SQLite имеет смысл, когда появляется серверный рантайм и нужно фильтровать, сортировать и надёжнее связывать услуги с технологиями в одном файле БД. Полноценный PostgreSQL и администрирование через IIS оправданы уже при многопользовательском

редактировании и централизованном сопровождении - это следующий уровень, выходящий за рамки утверждённой одностраничной концепции. В выполненном проекте выбран статический HTML/CSS/JS с localStorage только для темы оформления; SQLite и JSON рассмотрены как теоретические направления развития каталога проектов и услуг.

2 ПРОЕКТИРОВАНИЕ САЙТА-ВИЗИТКИ

2.1 Требования к функциональности и качеству

Перед сборкой интерфейса зафиксирован перечень ожидаемых возможностей:

1. показать фото, ФИО, роль и короткое описание умений;
2. вывести места работы и образование;
3. перечислить ключевые технологии;
4. дать анонс собственного SaaS и ссылку на него;
5. обеспечить быстрый выход на почту, GitHub и Telegram;
6. позволить менять светлую и тёмную палитру с запоминанием выбора;
7. автоматически считать срок работы «по настоящее время»;
8. уложить материал в макет «экран смартфона» так, чтобы основной каркас не требовал прокрутки.

К качеству предъявлялись требования лаконичного внешнего вида, быстрого открытия, понятных элементов управления, готовности к выкладке и базовой поисковой подготовки (title, description, canonical, Open Graph, sitemap).

Условие «один экран» заставило собрать сведения мозаикой внутри рамки телефона. Такой каркас усиливает ощущение мобильного формата и делает композицию собранной.

2.2 Выбор технологического стека

Реализация опирается на HTML5, CSS3 и JavaScript без серверных каркасов. Выбор следует из характера задачи: тексты почти не меняются, реакции происходят локально, а выкладка на обычный хостинг должна оставаться простой.

Разметка отвечает за смысл страницы: главный заголовок с именем, заголовки блоков, контактные ссылки и служебные метаданные. Стили строят сетку, неоновую эстетику, темы через CSS-переменные и движения элементов. Сценарии отвечают за стартовый экран загрузки, подсчёт стажа, смену темы и лёгкий параллакс фона.

Отказ от серверной СУБД на текущем шаге не уводит работу в сторону от дисциплины. Напротив, в тексте явно показано:

– где хватает файлов под контролем ОС;

– где уместно браузерное хранилище;

– когда без полноценной базы уже не обойтись (учётки, CMS, комментарии, поток событий).

В перспективе можно добавить API и таблицу проектов, чтобы обновлять сведения без правки HTML. Для учебной цели хватает статической поставки, и она проще для демонстрации идей курса.

2.3 Информационная архитектура и интерфейс

Страница собрана из одной мозаики блоков:

– верхняя полоса - личность, статус, контакты;

– опыт - две актуальные позиции с датами;

– образование - вуз, направление, ссылка на сайт;

– навыки - набор компактных «плашек»;

– SaaS - краткий анонс DigiGu.

Визуально всё помещено в корпус смартфона с узнаваемыми деталями: зона Dynamic Island и нижняя полоска Home. В «островке» расположена кнопка «ТЕМА», поэтому смена оформления находится на видном месте.

Цвет помогает читать карту блоков: опыт подсвечен розовым, учёба - синим, навыки - оранжевым, SaaS - зелёным. Даты двух мест работы дополнительно различаются песочным и виноградным тонами, а сами карточки слегка тонированы фоном.

На узких экранах media-запросы меняют сетку навыков, укорачивают длинные подписи и перестраивают шапку. Так выдерживается логика mobile-first.

Внешний вид опубликованной одностраничной визитки на GitHub Pages показан на рисунке 1. Макет в виде «экрана» устройства собирает блоки

идентичности, опыта, образования, навыков и анонса SaaS в одном кадре без обязательной прокрутки основного каркаса.



Рисунок 1 – Сайт-визитка на GitHub Pages (тёмная тема)

Та же визитка в светлой теме оформления показана на рисунке 2. Переключение выполняется кнопкой «ТЕМА» и запоминается в localStorage, поэтому после обновления страницы выбранная палитра сохраняется.

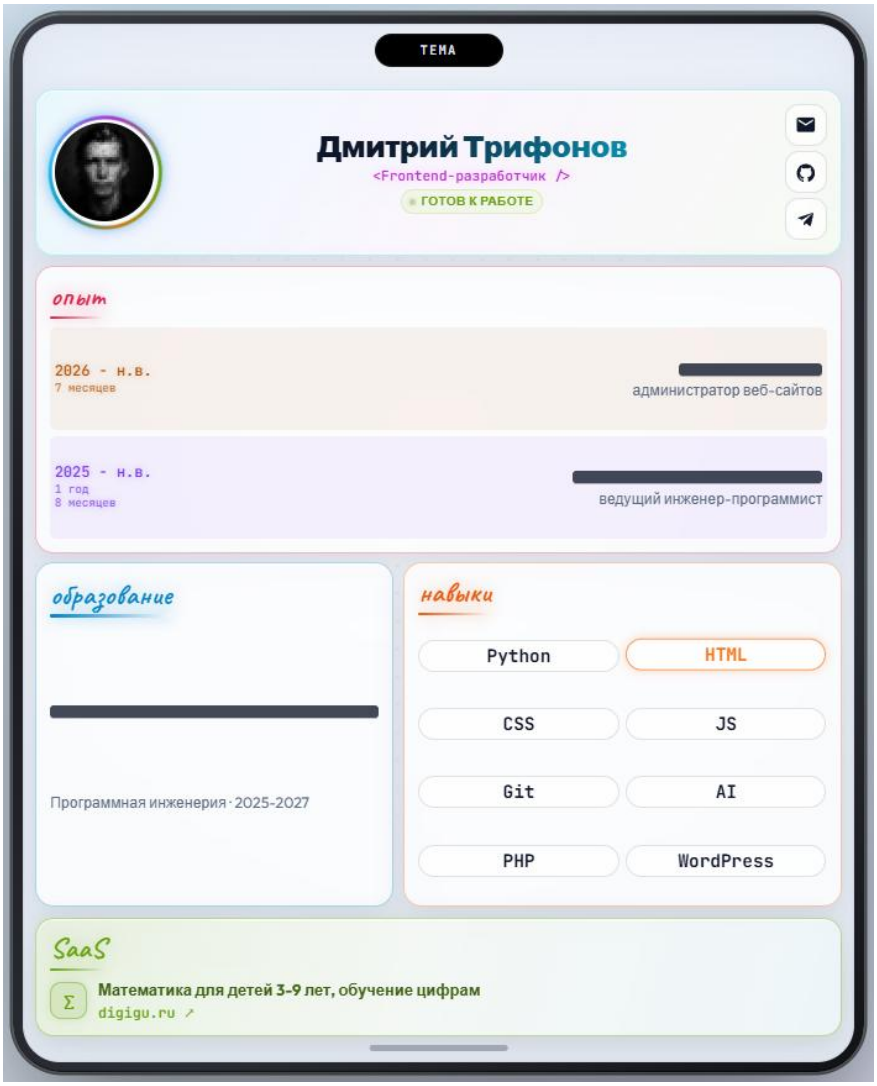


Рисунок 2 – Сайт-визитка на GitHub Pages (светлая тема)

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РАЗВЁРТЫВАНИЕ

3.1 Структура проекта и основные компоненты

Собранный результат - набор статических файлов:

– index.html содержит разметку, стили и клиентские сценарии;

– me.jpg хранит портрет;

– favicon.ico задаёт значок вкладки браузера;

– при публикации на GitHub Pages каталог репозитория отдаётся как статический сайт (в минимальном комплекте сейчас index.html, me.jpg и favicon.ico; служебные robots.txt и sitemap.xml относятся к боевой доводке и описаны в п. 3.3).

Соединение представления и логики в одном index.html ускоряет перенос: достаточно скопировать каталог в публичную область хостинга. Для учебного объёма это приемлемо; в промышленной практике стили и скрипты чаще выносят и собирают отдельно.

Среди заметных решений:

1. палитры через CSS-переменные и атрибут data-theme;

2. запоминание темы в localStorage ключом kyr2-theme;

3. подсчёт стажа из data-start формата MM.YYYY с русскими формами слов;

4. поочерёдная подсветка навыков, которая замирает при наведении;

5. SEO-описания и JSON-LD сущностей Person и WebPage.

Фрагмент разметки index.html, задающий каркас «iPhone-экрана», кнопку смены темы в зоне Dynamic Island и мозаику блоков, приведён на рисунке 3.

index.html — разметка каркаса визитки

```
<!-- фрагмент index.html: каркас «экрана» и шапка -->
<main class="stage">
  <div class="iphone">
    <div class="iphone-screen" id="iphoneScreen">
      <button type="button" class="dynamic-island"
              id="themeToggle" aria-label="Сменить тему">
        <span class="theme-label">ТЕМА</span>
      </button>
      <div class="mosaic">
        <header class="tile tile--header">
          <h1>Дмитрий Трифонов</h1>
          <p class="role">&lt;Frontend-разработчик /&gt;</p>
          ...
        </header>
        <!-- блоки опыта, образования, навыков, SaaS -->
      </div>
    </div>
  </div>
</main>
```

Рисунок 3 – Фрагмент разметки каркаса визитки в index.html

3.2 Взаимодействие с операционной средой хостинга

Ресурс доступен по адресу https://trifonix.github.io/trifonix-personal-website-prototype/. Площадка GitHub Pages даёт окружение статической публикации из репозитория: файлы визитки размещаются в каталоге проекта, а платформа обеспечивает выдачу по HTTPS.

С позиции ОС это модель статического хостинга: ядро системы и веб-сервер предоставляет площадка, а автор контролирует состав публичных файлов и их актуальность в репозитории. Подобная модель обычна для чисто статических персональных страниц.

Внутри index.html уже заданы: title и description страницы; указания robots/googlebot; канонический адрес (canonical); Open Graph и Twitter Card для превью в мессенджерах и соцсетях; JSON-LD сущностей Person и WebPage для машиночитаемого описания автора и страницы. Это и есть фактическая

поисковая подготовка прототипа; файлы robots.txt и sitemap.xml относятся к следующей боевой доводке (п. 3.3).

3.3 Служебные файлы robots.txt и sitemap.xml на боевом сайте

В минимальном учебном каталоге визитки достаточно трёх файлов: index.html, me.jpg и favicon.ico. Они уже обеспечивают показ страницы на GitHub Pages. Файлы robots.txt и sitemap.xml не обязательны для локального прототипа, но на реальном боевом сайте их обычно добавляют как часть подготовки к индексации и как элемент «пути поставки» публичного ресурса.

Файл robots.txt размещают в корне сайта (для GitHub Pages — в корне публикуемого репозитория или каталога docs/gh-pages, откуда отдаётся корень пути). Типовое содержимое для открытой визитки разрешает обход и указывает адрес карты сайта, например: User-agent: *; Allow: /; Sitemap: https://trifonix.github.io/trifonix-personal-website-prototype/sitemap.xml. После коммита и публикации площадка начинает отдавать файл по адресу .../robots.txt без дополнительной настройки сервера: для статического хоста это обычный текстовый файл в том же каталоге, что и index.html.

Файл sitemap.xml кладут рядом и описывают в нём канонические URL страниц. Для текущей одностраничной визитки достаточно одной записи url с loc на главный адрес, по желанию — lastmod и changefreq. Если позже появятся отдельные страницы проектов (как в теоретическом сценарии с JSON/SQLite), в карту добавляют их адреса тем же способом: правка XML в репозитории и повторная публикация. Поисковые системы подхватывают карту через ссылку из robots.txt либо через кабинет вебмастера.

На боевом контуре процедура выглядит так: 1) создать robots.txt и sitemap.xml в корне проекта; 2) прописать в Sitemap абсолютный HTTPS-адрес главной и при необходимости внутренних страниц; 3) закоммитить и дождаться публикации GitHub Pages (или скопировать файлы в document root на VPS/shared-хостинге); 4) проверить в браузере открытие /robots.txt и /sitemap.xml; 5) при желании отправить карту в инструменты вебмастера.

Отдельно от файлов остаются meta-теги в index.html (robots, canonical, Open Graph, JSON-LD): они уже есть в визитке и дополняют, а не заменяют карту сайта.

Таким образом, отсутствие robots.txt и sitemap.xml в текущей папке прототипа не противоречит теме работы: это следующий шаг боевой доводки статического сайта. Для дисциплины важно, что оба файла — обычные объекты файловой системы хостинга, которые ОС и веб-сервер отдают так же, как HTML и изображения, без СУБД и без серверного каркаса.

3.4 Клиентское хранение настроек и вычисляемые данные

Смена темы не ходит на сервер. При открытии страницы сценарий читает запись из localStorage и применяет палитру до показа основного интерфейса, чтобы уменьшить вспышку «чужого» цвета. Нажатие «ТЕМА» обновляет и экран, и сохранённое значение.

На рисунке 4 показан фрагмент клиентского сценария из того же index.html: расчёт подписи стажа по атрибуту data-start и сохранение выбранной темы в localStorage ключом kyr2-theme.

index.html — тема (localStorage) и стаж

```
// фрагмент index.html: тема и расчёт стажа
document.querySelectorAll('.job-period[data-start]').forEach((el) => {
  const tenure = el.querySelector('.job-tenure');
  tenure.replaceChildren(
    ...tenureFromStart(el.dataset.start).map((line) => {
      const span = document.createElement('span');
      span.textContent = line;
      return span;
    })
  );
});

const themeKey = 'kyr2-theme';
function applyTheme(theme) {
  const next = theme === 'light' ? 'light' : 'dark';
  document.documentElement.setAttribute('data-theme', next);
  try { localStorage.setItem(themeKey, next); } catch (e) {}
}
applyTheme(currentTheme());
themeToggle.addEventListener('click', () => {
  applyTheme(currentTheme() === 'light' ? 'dark' : 'light');
});
```

Рисунок 4 – Фрагмент сценария темы и расчёта стажа в index.html

Интересный фрагмент CSS из index.html приведён на рисунке 5: палитры светлой и тёмной тем заданы через CSS-переменные на селекторах [data-theme="dark"] и [data-theme="light"], а фон и цвет текста страницы берутся из var(--bg) и var(--ink). Так одна и та же вёрстка переключается без дублирования правил для каждого блока.

index.html — CSS: темы через переменные data-theme

```
/* фрагмент index.html: CSS-переменные двух тем */
[data-theme="dark"] {
  --bg: #0a0b12;
  --ink: #f1f5f9;
  --cyan: #22d3ee;
  --magenta: #e879f9;
  --lime: #a3e635;
  --tile-shadow: 0 0.5rem 1.5rem rgba(0,0,0,.28);
}

[data-theme="light"] {
  --bg: #dce3ef;
  --ink: #0f172a;
  --cyan: #0891b2;
  --magenta: #c026d3;
  --lime: #65a30d;
  --tile-shadow: 0 0.45rem 1.2rem rgba(15,23,42,.1);
}

html, body {
  height: 100dvh;
  overflow: hidden;
  background: var(--bg);
  color: var(--ink);
}

.iphone-screen {
  background: radial-gradient(
    ellipse 90% 70% at 50% 40%,
    var(--bg-mid) 0%, var(--bg-deep) 75%
  );
  box-shadow: var(--screen-inset);
}
```

Рисунок 5 – Фрагмент CSS: переменные светлой и тёмной тем в index.html

Срок работы считается в браузере по системной дате устройства. Для верхней карточки взята точка старта 04.2026, для нижней - 03.2025. Подпись выводится под «н.в.» двумя строками: годы и месяцы. Так демонстрируется работа со временем без СУБД: исходные даты лежат в разметке, а результат появляется в момент открытия.

Методически это подчёркивает простую мысль: не каждое вычисляемое поле обязано жить в таблице. Атрибуты HTML и клиентский расчёт уместны, пока нет нужды синхронно менять сведения для множества посетителей из одного центра.

3.5 Тестирование и результаты

Контроль включал:

– просмотр на широком и узком экране;

– проверку смены темы и её сохранения после обновления;

– сверку подсчёта стажа;

– работоспособность контактов и кнопки темы;

– наличие иконок вкладки и открытие боевого URL.

В итоге получена рабочая визитка с актуальными сведениями об авторе. Она фиксирует практику вёрстки, клиентских сценариев и выкладки и может служить и портфолио-элементом, и заготовкой для следующих персональных страниц.

ЗАКЛЮЧЕНИЕ

В курсовой работе рассмотрена разработка сайта-визитки фуллстек (фронтенд-фуллстек) разработчика в контексте современных операционных систем и подходов к хранению данных. Основные результаты состоят в следующем.

1. Показано, что сайт-визитка является эффективным средством профессиональной самопрезентации и может служить демонстрацией компетенций веб-разработчика.

2. Установлено, что даже статический веб-ресурс опирается на клиент-серверную модель и сервисы операционной системы хостинга: файловую систему, веб-сервер, сетевой стек и политики доступа.

3. Проанализированы варианты хранения данных. Для контента визитки достаточно файлового размещения; для пользовательских настроек интерфейса целесообразно клиентское хранилище localStorage; серверная СУБД требуется при появлении многопользовательской динамики и централизованного управления контентом.

4. Спроектирован и реализован одностраничный адаптивный сайт с блоками опыта, образования, навыков и проекта, а также с переключением темы и автоматическим расчётом стажа.

Выполнено развёртывание на GitHub Pages по адресу https://trifonix.github.io/trifonix-personal-website-prototype/. В index.html подготовлены конкретные средства поисковой и социальной разметки: title и meta description, meta robots/googlebot, keywords, link rel="canonical", Open Graph (og:title, og:description, og:image, og:url и др.), карточка Twitter (twitter:card/title/description/image) и два блока JSON-LD (Person и WebPage). Отдельные файлы robots.txt и sitemap.xml в прототипе ещё не добавлены — порядок их подключения на боевом сайте описан в п. 3.3.

Практическая значимость подтверждается наличием опубликованного результата. Дальнейшее развитие может включать серверный API, базу данных проектов, многоязычность и сбор обезличенной статистики посещений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Никсон Р. Создаём динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5. – СПб.: Питер, 2021. – 816 с.
2. Таненбаум Э., Бос Х. Современные операционные системы. – 4-е изд. – СПб.: Питер, 2020. – 1120 с.
3. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы. – СПб.: Питер, 2020. – 1008 с.
4. Дейт К. Дж. Введение в системы баз данных. – М.: Вильямс, 2019. – 1328 с.
5. Документация MDN Web Docs. Window.localStorage [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage (дата обращения: 10.09.2026).
6. Документация MDN Web Docs. HTML: HyperText Markup Language [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/HTML (дата обращения: 10.09.2026).
7. Спецификация Schema.org. Person [Электронный ресурс]. – Режим доступа: https://schema.org/Person (дата обращения: 10.09.2026).
8. GitHub Docs. GitHub Pages [Электронный ресурс]. – Режим доступа: https://docs.github.com/en/pages (дата обращения: 10.11.2026).
9. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. – СПб.: Питер, 2020. – 366 с.
10. Фримен Э., Робсон Э. Изучаем HTML, CSS и JavaScript. – СПб.: Питер, 2021. – 720 с.